*The Problem of Tight Coupling :-

In oop, tight coupling occurs when one class is directly dependent on the concrete implementation of another class. This makes the code rigid and hard to maintain.

Example of a Tightly Coupled Design →

Imagine a OrderService class that needs to send a notification whenever an order is placed. If the OrderService directly creates an Object of EmailService inside its method :

- EmailService notification = new EmailService();

- Issue → The OrderService is now "married" to EmailService. If you later decide to switch to SMS Service or any

Date.....

other service, you must modify the code inside OrderService class.

Why this is bad practice → Directly creating dependencies inside your business logic violates several core software design principles:

- Single Responsibility Principle (SRP) → An OrderService should only worry about processing orders, not creating notification objects. It is taking extra responsibility (acting as a factory).

- Open-Closed Principle → Your code should be open for extension but closed for modification. With tight coupling, adding a new notification method requires you to open up and change existing, working code in OrderService.

Real-world analogy → Think of it like deciding to travel from Delhi to Chandigarh. If you mandate that you must take a specific bus, with a specific driver, from a specific company, you are tightly coupled to these details.

Date.....

Achieving Loose Coupling with Interfaces :-

To move away from tight coupling, we transition from depending on concrete classes to depending on Interfaces.
We create a NotificationService interface :-

```
public interface NotificationService {
    void sendNotification();
}
```

Now, any specific implementation (like Email, SMS, etc.) must implement this interface and override this method.
In OrderService, we change →

```
NotificationService notification = new EmailService();
```

Why this is still not enough? →

While we are now using an interface type, OrderService is 'still creating the object' internally. This means it is still responsible for lifecycle of its dependencies.
This makes the code more flexible, but it still doesn't solver the problem of Open-Closed Principle.

Date.....

* Dependency Injection :-

To solve the issue where OrderService is responsible for creating its own dependencies, we introduce Dependency Injection (DI).
It is a software design pattern where an object receives its dependencies from an external source rather than creating them internally.

This is commonly done through a Constructor.

- The Change → Add a constructor to OrderService that accepts a NotificationService object.
- The Result → OrderService no longer cares which specific notification implementation it receives. It simply use whatever is given.

Benefits →

- Independence: OrderService is truly decoupled. It follows Open-Closed Principle because you can switch notification types without touching a single line in OrderService.

- Improved Unit Testing: We can now easily test OrderService by injecting a 'fake' or 'Dummy' implementation of interface. This allows us to test the

Date.....

business logic without actually sending real emails or notifications during testings.

Types of Injection →

- Constructor: Passing the dependency through class constructor.
- Setter: Passing the dependency through a setter method.

* Inversion of Control & It is a design principle where the 'flow of control' in a program is reversed.

IoC is the Principle: It is the high-level design goal that states a class should not manage its own dependencies.

DI is the Technique: It is the specific approach used to implement IoC. By injecting dependencies through constructors or setters, you achieve the inversion of control.

☆ IoC is a broad principle, not a single pattern. DI is just one of those patterns that implement it. Like - Template Method, Factory, Observer, etc.

Date.....

* IOC Container :-

It is the core of Spring framework responsible for instantiating, configuring, and managing application objects known as Beans.
It :-

- creates objects (Beans) for you.
- Manages their lifecycle.
- Automatically wire (inject) the dependencies between them.

This 'magic' is possible because Spring uses Java Reflection API to inspect classes and instantiate them dynamically at runtime.

→ Setting up IOC Container :-

1. Dependency Setup → You need 'spring-context' dependency in your pom.xml.
This provides core IOC functionality.

2. Marking Beans with @Component → To tell spring which classes it should manage, you annotate them with @Component.
Ex → Placing @Component above OrderService and PaymentService classes make them as candidates for Spring to instantiate and manage.

3. Configuring the Container → Spring uses an "ApplicationContext" as interface for IOC Container.

Date.....

- Create a configuration class : Use the @Configuration annotation on a class (e.g., AppConfig).
- Enable Scanning : Add @ComponentScan to this class. This tells Spring which packages to scan to find classes annotated with @Component.
- Initialize : In your main method, initialize the container using ~~[Application Config]~~ [AnnotationConfigApplicationContext] by passing in [AppConfig] class.

4. Fetching Beans → Once the container is initialized, you can retrieve instances of your objects from the context using the getBean() method.

OrderService order = context.getBean(OrderService.class);

Once your beans are managed by Spring, you can inject dependencies between them. Spring provides 3 ways to achieve this :-

→ Ways to Inject Dependencies :-

1. Constructor Injection → Dependencies are provided through the class constructor. If you only have one constructor, Spring automatically wires

Teacher's Sign.....

Date.....

the dependency without needing @Autowired. This approach is preferred because it allows you to make dependencies 'final', ensures the object is fully initialized when created, and makes unit testing much easier.

2. Setter Injection → Dependencies are set via setter methods annotated with @Autowired. This is useful for optional dependencies or when you need to reconfigure an object after its initial creation.

3. Field Injection → Dependencies are injected directly into private fields using @Autowired. While convenient, this is generally not recommended as it makes testing difficult and hides dependencies.

* How Spring Manages Beans :-

When you call new AnnotationConfigApplicationContext(AppConfig.class), the following happens internally:

1. Container Startup → Spring starts the ApplicationContext container.

2. Configuration Reading → Spring reads your configuration class to identify rules and packages to scan.

Date.....

3. Component Scanning → It finds all classes marked with @Component.

4. Bean Definitions → Spring creates Bean Definitions — metadata objects that store information about how to create bean (eg. class name, dependencies).

5. Instantiation → Spring instantiates the beans. If a bean has dependencies (like OrderService needing PaymentService), it resolves and injects those dependencies in correct order.

6. Ready for Use → The beans are fully prepared and stored in IOC Container, ready to be retrieved and via context.getBean().

** Why use Configuration class?
we use a separate AppConfig class instead of putting everything in main class to keep our application entry point clean and separate 'Configuration' (how objects are built) from 'Execution' (running the logic).

Date.....

As application grow, you often need to manage multiple implementations of same interface or handle classes that are parts of third-party libraries.

* Handling Multiple Beans :-

- Problem → If you have an interface (e.g. PaymentService) with two implementations (e.g. CardPayment & UPIPayment) both marked with @Component, Spring won't know which one to inject when OrderService asks for PaymentService.

- @Primary → Use this on a class to mark it as the default implementation. If Spring finds multiple candidates, it will choose the one annotated with @Primary.

- @Qualifier → Use this at injection point (e.g. OrderService constructor) to specify exactly which bean name to use. (e.g. @Qualifier("cardPayment"));

* @Bean Annotation :-

While @Component is great for your own classes, sometimes you cannot modify the source code to add annotations (e.g. classes from a library).

Date.....

- How it works → You define a method within a @Configuration class that returns object you want managed. Annotate that method with @Bean.

- Spring's Role → Spring will execute this method, capture the returned object, and store it in IOC container as a bean.

- Difference → @Component is for auto-detection (Spring scans for it), while @Bean is for manual registration (you explicitly write the code to create the bean).

* Mixing @Bean and @Component?

- Flexibility → You are not limited to one approach. You can use @Component for classes you own and @Bean within your config class for 3-party classes or scenarios where you need more granular control over the object's process (e.g. passing specific arguments to a constructor).

- Dependency Injection in @Bean methods → When defining a @Bean, if creation of that object depends on another bean, you can simply pass required dependency as a parameter to your @Bean

Date.....

method. Spring will automatically resolve and inject the correct bean from context.

* Beanfactory Vs ApplicationContext :-

- Beanfactory → It is the simplest & most basic container in Spring. It provides basic features for bean creation and management and provides fundamental DI support. Beans are created lazily ; means instantiated only when requested.

- ApplicationContext → It is a more advanced and feature-rich container compared to Bean Factory. It extends Beanfactory and adds additional features such as event propagation, AOP support, and internationalization. Here, beans are created eagerly. It is the industry ready standard for enterprise applications.